

UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA

**FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS**

TALLER - GESTIÓN DE LA CALIDAD - PARTE 2
PUNTO 4: TÉCNICAS Y HERRAMIENTAS DE Auditoría DE PROCESOS
APLICADAS AL PROYECTO PETCARE

Proyecto: PetCare Plataforma
Empresa / Organización: PetCare

Equipo:
Harold Ricardo Alvarado Leandro
Juan José Forero Martin
Andrés Santiago Daza
Anthony Alejandro González Sánchez

1. INTRODUCCIÓN

El presente documento corresponde a la solución del Punto 4 del Taller de GESTIÓN de la Calidad - Parte 2, que solicita investigar y documentar al menos dos TÉCNICAS y herramientas de Auditoría de procesos aplicables al proyecto PetCare.

PetCare es una plataforma digital compuesta por una aplicación móvil y página web que conecta a dueños de mascotas con prestadores de servicios especializados (grooming, paseo, entrenamiento, guardería), ofreciendo contratación a domicilio, monitoreo en tiempo real y un sistema de reputación basado en calificaciones verificadas.

Stack tecnológico actualizado del proyecto:

- Frontend: React 19 + Vite
- Backend: Firebase (Firestore, Authentication)

Se han seleccionado dos TÉCNICAS de Auditoría de procesos ampliamente reconocidas en la industria del software y la GESTIÓN de calidad:

1. Checklist de Auditoría de Procesos de Software
2. revisión por Pares (Peer Review / Code Review) de Procesos de Software

2. técnica 1: CHECKLIST DE Auditoría DE PROCESOS DE SOFTWARE

2.1 Definición

Un checklist o lista de verificación es una herramienta estructurada que permite analizar cada detalle de la planificación y ejecución de una Auditoría. Contiene preguntas específicas que permiten conocer los procesos de documentación de la empresa, revisar los requisitos de la normativa aplicable (como ISO 9001) y verificar el cumplimiento de estándares de calidad en cada etapa del proceso. Los checklists no deben tener un formato general; por el contrario, deben adaptarse a los procesos de cada proyecto.

2.2 Objetivos de Aplicación en PetCare

- Verificar que los procesos de desarrollo de software (frontend con React 19 + Vite, backend con Firebase) cumplan con los estándares definidos en el Agile Inception Deck.
- Evaluar la implementación de controles de seguridad (Firebase Authentication, reglas de seguridad de Firestore, HTTPS) en cada fase del ciclo de vida del software.

- Confirmar que la documentación técnica (arquitectura cliente-servidor, modelo de datos Firestore, integración Firebase) esté actualizada y sea coherente con el código desarrollado.
- Identificar desviaciones o no conformidades en los procesos de GESTIÓN de calidad antes de que afecten el producto final.
- Generar evidencia documental objetiva para la toma de decisiones correctivas y preventivas.

2.3 Aplicación en el Proyecto PetCare

A continuación se presenta el checklist de Auditoría de procesos diseñado específicamente para el proyecto PetCare, organizado por Áreas críticas del desarrollo del software y adaptado al stack tecnologico React 19 + Vite / Firebase:

Área / Proceso	Pregunta de Auditoría	Cumple (Si / No / N/A)	Hallazgo / Observacion
GESTIÓN DE REQUISITOS	El Product Owner (Juan José Forero) ha priorizado el backlog inicial con historias de usuario validadas?		
GESTIÓN DE REQUISITOS	Los requisitos del MVP están alineados con el alcance definido en el NOT list?		
GESTIÓN DE REQUISITOS	Se han identificado y documentado los riesgos principales del proyecto?		
GESTIÓN DE REQUISITOS	El Agile Inception Deck está completo y aprobado por todos los stakeholders?		
ARQUITECTURA Y diseño	La arquitectura cliente-servidor con Firebase está documentada y validada?		
ARQUITECTURA Y diseño	El modelo de datos en Firestore está definido con colecciones y documentos aprobados?		
ARQUITECTURA Y diseño	Los wireframes/prototipos de interfaz (Figma o similar) están disponibles y validados?		
ARQUITECTURA Y diseño	Se ha definido la estrategia de despliegue con Firebase Hosting?		
DESARROLLO FRONTEND	El código fuente en React 19 + Vite sigue		

	los estándares de codificación definidos?		
DESARROLLO FRONTEND	La interfaz es responsive y se adapta a dispositivos móviles?		
DESARROLLO FRONTEND	Se han implementado validaciones del lado cliente en formularios de registro y solicitud de servicios?		
DESARROLLO FRONTEND	La integración con Firebase (Firestore, Auth) esta correctamente implementada con SDK de Firebase?		
DESARROLLO FRONTEND	Se ha implementado el manejo de estados y autenticación con Firebase Authentication?		
DESARROLLO FRONTEND	Se utilizan React Hooks modernos y el nuevo sistema de compilación de React 19?		
DESARROLLO BACKEND	Las reglas de seguridad de Firestore están configuradas para restringir acceso por roles (Dueño, Prestador, Admin)?		
DESARROLLO BACKEND	La autenticación con Firebase Authentication implementa verificación de email y control de sesiones?		
DESARROLLO BACKEND	El control de roles (Dueño, Prestador, Admin) esta implementado en las reglas de Firestore y en la lógica de la aplicación?		
DESARROLLO BACKEND	La lógica de negocio (GESTIÓN de servicios, solicitudes, calificaciones) esta correctamente implementada en Firestore?		
DESARROLLO BACKEND	Se han configurado Cloud Functions de Firebase para procesos server-side críticos?		
BASE DE DATOS	La base de datos Firestore esta correctamente modelada con colecciones (Usuarios, Mascotas, Servicios,		

	Solicitudes, Calificaciones)?		
BASE DE DATOS	Se han implementado reglas de seguridad de Firestore para validar integridad de datos?		
BASE DE DATOS	Existe un plan de respaldo y recuperacion de datos en Firestore?		
BASE DE DATOS	El acceso a Firestore esta restringido por Firebase Authentication y reglas de seguridad?		
SEGURIDAD	Se utiliza HTTPS obligatoriamente en todas las comunicaciones con Firebase?		
SEGURIDAD	Las contraseñas de usuarios se gestionan exclusivamente mediante Firebase Authentication?		
SEGURIDAD	Se han implementado reglas de Firestore para prevenir lecturas/escrituras no autorizadas?		
SEGURIDAD	Se ha realizado una Auditoría de seguridad basica de las reglas de Firestore?		
SEGURIDAD	Los API keys de Firebase están protegidos y no expuestos en el código fuente publico?		
CONTROL DE VERSIONES	El código fuente esta alojado en GitHub con control de versiones activo?		
CONTROL DE VERSIONES	Se han definido ramas (branches) para desarrollo, pruebas y produccion?		
CONTROL DE VERSIONES	Se realizan code reviews antes de fusionar cambios a la rama principal?		
PRUEBAS Y CALIDAD	Se han diseñado casos de prueba funcionales y de integración?		
PRUEBAS Y CALIDAD	Se han ejecutado pruebas de seguridad basicas de las reglas de Firestore?		
PRUEBAS Y CALIDAD	Los bugs criticos identificados han sido documentados y corregidos?		

PRUEBAS Y CALIDAD	Se ha validado la experiencia de usuario (UX) con usuarios piloto?		
DESPLIEGUE Y OPERACION	La plataforma esta desplegada en Firebase Hosting con configuración HTTPS?		
DESPLIEGUE Y OPERACION	Se ha configurado el monitoreo basico con Firebase Analytics y Firebase Crashlytics?		
DESPLIEGUE Y OPERACION	El dominio esta configurado y vinculado correctamente con Firebase Hosting?		
DESPLIEGUE Y OPERACION	Existe un plan de contingencia ante fallos del servicio de Firebase?		
GESTIÓN DE PROYECTO	Se han establecido reuniones de seguimiento (daily standups, sprint reviews)?		
GESTIÓN DE PROYECTO	El cronograma de 14 semanas se esta cumpliendo según lo planificado?		
GESTIÓN DE PROYECTO	Se ha documentado la retroalimentacion de los primeros usuarios y prestadores registrados?		
GESTIÓN DE PROYECTO	El presupuesto estimado (\$17.120.000 COP) se esta respetando?		

2.4 Beneficios de la técnica para PetCare

- Permite una revisión sistemática y exhaustiva de todos los procesos criticos del desarrollo del software con React 19 + Vite y Firebase.
- Facilita la identificación temprana de no conformidades en la configuración de Firebase (Auth, Firestore, Hosting) antes de que escalen a problemas mayores.
- Genera evidencia documental objetiva para Auditorías internas y externas.
- Ayuda a mantener la trazabilidad entre los requisitos del Inception Deck y su implementacion técnica en Firebase.
- Es una herramienta adaptable que puede reutilizarse en cada fase del proyecto (Inception, diseño, Desarrollo, Pruebas, Despliegue).

3. técnica 2: revisión POR PARES (PEER REVIEW / CODE REVIEW)

3.1 Definición

La revisión por pares (Peer Review) es un proceso sistematico mediante el cual un miembro del equipo de desarrollo revisa y evalua el trabajo de otro companero para detectar errores, mejorar la calidad del código y garantizar el cumplimiento de estándares tecnicos y de calidad. En el contexto del software, tambien se conoce como Code Review cuando se aplica especificamente al código fuente.

Este proceso es una técnica fundamental de Auditoría de procesos porque permite evaluar la calidad del producto intermedio (código, documentación, diseño) antes de que avance a la siguiente fase, reduciendo drasticamente el costo de correccion de defectos. según estudios de la industria, detectar un error durante una revisión por pares cuesta hasta 10 veces menos que detectarlo en pruebas de integración o en produccion.

3.2 Objetivos de Aplicación en PetCare

- Detectar defectos tecnicos (errores de lógica, vulnerabilidades de seguridad, malas prácticas de codificacion) en el código fuente React 19 + Vite y en la configuración de Firebase antes de su integración.
- Garantizar que el código desarrollado por cada miembro del equipo (Harold, Anthony, Andrés, Juan Jose) cumpla con los estándares definidos en la arquitectura cliente-servidor con Firebase.
- Verificar la consistencia entre la documentación técnica (modelo de datos Firestore, reglas de seguridad, integración Firebase Auth) y su implementacion real en el código.
- Fomentar el conocimiento compartido del código y la configuración de Firebase entre todos los miembros del equipo, reduciendo la dependencia de un solo desarrollador.
- Evaluar la adherencia a las mejores prácticas de seguridad (Firebase Authentication, reglas de Firestore, HTTPS, proteccion de API keys) definidas en el Inception Deck.
- Generar retroalimentacion constructiva que mejore las habilidades TÉCNICAS del equipo y eleve la calidad general del producto.

3.3 Tipos de revisión por Pares Aplicables a PetCare

Para el proyecto PetCare, se han seleccionado tres tipos de revisión por pares que cubren las necesidades especificas de un equipo compacto de cuatro personas con roles definidos y stack tecnologico React 19 + Vite / Firebase:

Tipo de revisión	Descripción	Aplicación en PetCare
revisión de Código (Code Review)	revisión técnica formal del código fuente por un compañero diferente al autor. Se analizan: lógica de negocio, estructura de código, seguridad, rendimiento y adherencia a estándares.	Anthony revisa el código React 19 + Vite de Andrés y viceversa. Harold revisa código crítico de ambos. Se aplica en cada Pull Request de GitHub antes de merge. Se verifica integración correcta con Firebase SDK.
revisión técnica (Technical Review)	Evaluación de aspectos técnicos del diseño, arquitectura y documentación. No solo se revisa código, sino también decisiones de diseño, modelo de datos Firestore y estrategias de despliegue.	Harold revisa las decisiones de arquitectura con Firebase, el modelo de datos en Firestore (colecciones y documentos) y la configuración de Firebase Hosting. Juan Jose valida que el diseño cumpla con los requisitos del Inception Deck.
Inspección Formal (Inspection)	revisión estructurada y rigurosa que sigue un protocolo definido. Se asignan roles específicos (autor, moderador, revisor, secretario) y se documentan todos los hallazgos.	Se aplica en hitos críticos del proyecto: antes del despliegue del MVP en Firebase Hosting, antes de la entrega de la fase de pruebas, y antes de la validación inicial con usuarios piloto en Tunja.

3.4 Proceso de revisión por Pares en PetCare

El siguiente proceso detalla paso a paso como se implementa la revisión por pares en el proyecto PetCare, desde la preparación hasta el cierre de la revisión:

Fase	Actividad	Responsable(s)	Entregable / Evidencia
1. PLANIFICACION	Definir el alcance de la revisión: que módulos, funcionalidades o configuraciones de Firebase serán revisados. Establecer criterios de aceptación y checklist de revisión.	Harold Alvarado (Lider de Desarrollo / Arquitecto)	Plan de revisión por Pares documentado con alcance, criterios y cronograma.
2. PREPARACION	El autor prepara el material a revisar: código limpio en React 19 + Vite, configuración de Firebase actualizada, descripción de cambios y contexto del negocio. Crea un Pull Request en GitHub con descripción detallada.	Autor del código (Andrés Daza o Anthony González)	Pull Request en GitHub con descripción de cambios, referencias a historias de usuario y documentación adjunta.
3. ASIGNACION	El líder asigna el revisor apropiado según el área: Frontend (Andrés) -> Revisor (Anthony) o Líder (Harold). Backend/Firebase (Anthony) -> Revisor (Andrés) o Líder (Harold).	Harold Alvarado	Asignación documentada en GitHub Projects o Jira con fecha límite de revisión.

	documentación -> Revisor Product Owner (Juan Jose).		
4. EJECUCION DE LA revisión	El revisor analiza el código/documentación usando el checklist de calidad. Verifica: lógica de negocio, seguridad (Firebase Auth, reglas Firestore, proteccion de API keys), estándares de código React 19, pruebas unitarias, documentación. Deja comentarios en GitHub.	Revisor asignado (según area)	Comentarios en GitHub Pull Request con observaciones clasificadas: CRITICO, MAYOR, MENOR, SUGERENCIA.
5. RETROALIMENTACION	El revisor y el autor se reúnen (virtual o presencial) para discutir los hallazgos. El autor explica decisiones de diseño y el revisor justifica sus observaciones. Se llega a acuerdos sobre correcciones necesarias.	Autor + Revisor	Acta de reunión de revisión (opcional) o thread de discusión en GitHub documentado.
6. CORRECCION	El autor implementa las correcciones acordadas. Actualiza el código React 19 + Vite, configuración de Firebase y pruebas según los comentarios del revisor. Responde a cada comentario en GitHub indicando la acción tomada.	Autor del código	Commits adicionales en el Pull Request con referencias a los comentarios corregidos.
7. verificación	El revisor verifica que las correcciones resuelven los hallazgos sin introducir nuevos defectos. Revisa los nuevos commits y aprueba o solicita ajustes adicionales.	Revisor asignado	Aprobación formal en GitHub (Approve) o solicitud de cambios adicionales (Request Changes).
8. CIERRE	Una vez aprobada, el líder (Harold) o el autor realiza el merge a la rama principal. Se archiva la evidencia de la revisión. Se actualiza el registro de revisiones.	Harold Alvarado o Autor	Merge completado en GitHub, registro de revisión en documento de trazabilidad de calidad.

3.5 Checklist de revisión por Pares para PetCare

A continuación se presenta el checklist específico que debe utilizar cada revisor durante la revisión por pares en el proyecto PetCare. Este checklist está alineado con el stack

tecnologico React 19 + Vite / Firebase y los estándares de calidad definidos en el Inception Deck:

Categoría de revisión	Criterio a Verificar	Estado (OK / NC / N/A)
FUNCIONALIDAD	El código implementa correctamente la funcionalidad descrita en la historia de usuario?	
FUNCIONALIDAD	Se han manejado todos los casos de borde (edge cases) y escenarios de error?	
FUNCIONALIDAD	La lógica de negocio es coherente con las reglas definidas en el Inception Deck?	
SEGURIDAD	Las reglas de seguridad de Firestore restringen el acceso según el rol del usuario (Dueño, Prestador, Admin)?	
SEGURIDAD	La autenticación con Firebase Authentication implementa verificación de email y control de sesiones?	
SEGURIDAD	Los API keys de Firebase están protegidos mediante variables de entorno y no expuestos en el código fuente?	
SEGURIDAD	Se validan y sanitizan todas las entradas de usuario antes de enviarlas a Firestore?	
SEGURIDAD	Las comunicaciones con Firebase usan HTTPS obligatoriamente?	
SEGURIDAD	No se exponen datos sensibles (tokens, UIDs) en logs o mensajes de error del cliente?	
CALIDAD DE CODIGO	El código React 19 sigue los estándares de nomenclatura y estructura definidos (ESLint, Prettier)?	
CALIDAD DE CODIGO	Las funciones y componentes React tienen una única responsabilidad (Single Responsibility Principle)?	
CALIDAD DE CODIGO	El código es legible, con comentarios significativos donde sea necesario?	
CALIDAD DE CODIGO	No hay código duplicado (DRY - Don't Repeat Yourself)?	
CALIDAD DE CODIGO	Se utilizan React Hooks modernos (useState, useEffect, useContext) y el nuevo sistema de compilación de React 19?	
ARQUITECTURA	El código respeta la separación de responsabilidades entre componentes React, lógica de negocio y servicios Firebase?	
ARQUITECTURA	Las integraciones con Firebase (Auth, Firestore, Storage,	

	Hosting) usan el SDK oficial y siguen las mejores prácticas?	
ARQUITECTURA	El modelo de datos en código coincide con el modelo de colecciones y documentos de Firestore aprobado?	
ARQUITECTURA	Se han configurado Cloud Functions de Firebase para procesos que requieren lógica server-side?	
PRUEBAS	Se han escrito pruebas unitarias para las funciones críticas con Vitest (incluido en Vite)?	
PRUEBAS	Las pruebas cubren casos de éxito y casos de error?	
PRUEBAS	Todas las pruebas existentes pasan sin errores después de los cambios (npm run test)?	
documentación	El código incluye documentación de funciones complejas (JSDoc) y componentes React?	
documentación	El Pull Request incluye descripción clara de los cambios, motivación y referencias a historias de usuario?	
documentación	Se ha actualizado la documentación técnica si los cambios afectan la arquitectura Firebase?	
RENDIMIENTO	Las consultas a Firestore están optimizadas (índices compuestos, límites de resultados)?	
RENDIMIENTO	No hay bucles innecesarios o renderizados excesivos en componentes React?	
RENDIMIENTO	Se utilizan TÉCNICAS de lazy loading o code splitting de Vite para optimizar el bundle?	
UX/UI	La interfaz es responsive y se adapta a dispositivos móviles (media queries, flexbox/grid)?	
UX/UI	Los mensajes de error son claros y orientan al usuario a resolver el problema?	
UX/UI	La navegación y flujo de usuario es intuitivo y coherente con los wireframes aprobados?	

3.6 Matriz de Asignación de revisiones por Pares

La siguiente matriz establece quien revisa a quien en el equipo de PetCare, según el Área de trabajo y la criticidad del componente:

Autor del Trabajo	Rol del Autor	Revisor Asignado	Tipo de revisión / Frecuencia
Andrés Santiago Daza	Desarrollador Frontend (React 19 + Vite)	Anthony Alejandro González	Code Review de cada Pull Request. revisión

			de integración Firebase SDK.
Anthony Alejandro González	Desarrollador Backend / Firebase	Andrés Santiago Daza	Code Review de cada Pull Request. revisión de reglas Firestore y configuración Firebase.
Andrés Santiago Daza	Desarrollador Frontend	Harold Ricardo Alvarado	revisión técnica de arquitectura frontend, componentes React 19 y decisiones de diseño UI/UX.
Anthony Alejandro González	Desarrollador Backend / Firebase	Harold Ricardo Alvarado	revisión técnica de arquitectura Firebase, modelo de datos Firestore y seguridad.
Harold Ricardo Alvarado	Arquitecto / Lider de Desarrollo	Juan José Forero	revisión de documentación de arquitectura, riesgos y decisiones TÉCNICAS.
Juan José Forero	Product Owner / Lider de Proyecto	Harold Ricardo Alvarado	revisión de requisitos, backlog y alineación con objetivos de negocio.
Cualquier miembro	Cualquier rol	Todo el equipo	Inspección Formal en hitos críticos: fin de fase de diseño, fin de Desarrollo MVP, antes de Despliegue en Firebase Hosting.

3.7 Registro de revisiones por Pares

Para garantizar la trazabilidad y evidencia documental de las revisiones por pares, se debe mantener un registro con la siguiente información:

- ID de la revisión (ej: REV-001, REV-002)
- Fecha de la revisión
- Autor del trabajo revisado
- Revisor asignado
- Tipo de revisión (Code Review, revisión técnica, Inspección Formal)
- Componente / Módulo revisado (ej: Módulo de autenticación Firebase, Componente de Servicios React, Reglas Firestore)
- Número de hallazgos identificados (Críticos, Mayores, Menores, Sugerencias)
- Número de hallazgos corregidos
- Estado de la revisión (En Progreso, Aprobada, Rechazada, Pendiente de Correcciones)
- Observaciones generales
- Enlace al Pull Request en GitHub

3.8 Beneficios de la revisión por Pares para PetCare

- Detecta defectos hasta 10 veces mas barato que en pruebas de integración o produccion, reduciendo costos de corrección en el stack React 19 + Vite / Firebase.
- Mejora la calidad del código fuente al garantizar que cumple con estándares de seguridad (Firebase Auth, reglas Firestore, protección de API keys) y arquitectura cliente-servidor.
- Distribuye el conocimiento técnico de Firebase y React 19 entre todos los miembros del equipo, evitando la dependencia de un solo desarrollador.
- Fomenta una cultura de calidad y mejora continua, donde cada miembro del equipo es responsable de la calidad del producto final.
- Genera evidencia documental objetiva de que los procesos de control de calidad se están ejecutando según lo planificado.
- Facilita el cumplimiento de estándares internacionales como ISO 9001 e ISO/IEC 25010 en el desarrollo de software con tecnologías modernas como React 19 y Firebase.

4. CONCLUSIONES

Las dos TÉCNICAS de Auditoría de procesos presentadas -Checklist de Auditoría de Procesos de Software y revisión por Pares (Peer Review / Code Review)- son herramientas complementarias y altamente efectivas para garantizar la calidad del proyecto PetCare con stack tecnologico React 19 + Vite / Firebase.

El checklist permite una revisión sistematica y detallada de cada Área critica del desarrollo del software, asegurando que los estándares de calidad, seguridad y documentación definidos en el Agile Inception Deck se cumplan en cada fase del proyecto, incluyendo la correcta configuración de Firebase Authentication, Firestore y Firebase Hosting.

La revisión por pares proporciona un mecanismo de control de calidad continuo durante el desarrollo, permitiendo detectar defectos tecnicos, vulnerabilidades de seguridad y desviaciones de arquitectura en el código React 19 y la configuración de Firebase antes de que se integren al producto final.

La aplicacion conjunta de ambas TÉCNICAS permite establecer un sistema de GESTIÓN de calidad robusto para PetCare, alineado con las mejores prácticas de la industria del software y los principios de mejora continua establecidos en normas como ISO 9001 e ISO/IEC 25010.

Se recomienda implementar estas TÉCNICAS de manera periodica: el checklist al finalizar cada fase del proyecto (Incepcion, diseño, Desarrollo, Pruebas y Despliegue) y la revisión por pares de forma continua durante todo el ciclo de desarrollo, en cada Pull Request y en hitos criticos del proyecto, garantizando asi la calidad del MVP de PetCare desplegado en Firebase Hosting para la ciudad de Tunja.

5. REFERENCIAS BIBLIOGRAFICAS

3. AuditBrain. (2024). 8 herramientas para llevar a cabo una optima Auditoría interna. Recuperado de: <https://auditbrain.com/8-herramientas-para-llevar-a-cabo-una-optima-Auditoría-interna/>
4. Bambu Mobile. (2024). TÉCNICAS de revisión de software. Recuperado de: <https://bambu-mobile.com/TÉCNICAS-de-revisión-de-software/>
5. Ciberso. (2025). Que es una Auditoría de aplicaciones y software? Recuperado de: <https://ciberso.com/blog/Auditorías-aplicaciones-y-software/>
6. Consejo de Auditoría Interna General de Gobierno de Chile. (2015). Documento Tecnico N 75: TÉCNICAS y Herramientas para el Control de Procesos y la GESTIÓN de la Calidad. Santiago, Chile.
7. Escuela Europea de Excelencia. (2018). 5 herramientas de Auditoría interna de calidad. Recuperado de: <https://www.escuelaeuropeaexcelencia.com/2018/06/5-herramientas-de-Auditoría-interna-de-calidad/>
8. Firebase Documentation. (2026). Security Rules, Authentication, Firestore. Recuperado de: <https://firebase.google.com/docs>
9. React Documentation. (2026). React 19. Recuperado de: <https://react.dev>
10. SafetyCulture. (2021). Herramientas de GESTIÓN de calidad. Recuperado de: <https://safetyculture.com/es/temas/herramientas-de-GESTIÓN-de-calidad>

11. The Codest. (2024). Guia completa de herramientas y TÉCNICAS de Auditoría informática. Recuperado de:
<https://thecodest.co/es/blog/guia-completa-de-herramientas-y-TÉCNICAS-de-Auditoría-informatica/>
12. Universidad Politécnica de Madrid. GESTIÓN, Control y Garantía de la Calidad del Software. Recuperado de:
<https://oa.upm.es/89291/1/calidad.pdf>